Contents

1 Introduction **3**
- 1.1 Design Philosophy . . . 3
- 1.2 Scope . . . 3

2 VRAM Estimation Model **3**
- 2.1 Model Weight Memory . . . 3
- 2.2 KV Cache Memory . . . 4
  - 2.2.1 Attention Architecture Variants . . . 4
  - 2.2.2 Fallback Estimation . . . 5
- 2.3 Framework Overhead . . . 5

3 RAM Overflow / Offloading **5**
- 3.1 The Bus Wall ("Le Mur du Bus") . . . 6
- 3.2 Improved RAM Offload Performance Model . . . 6
- 3.3 VRAM Priority Allocation & Offloading Regimes . . . 6
  - 3.3.1 N1: VRAM Priority Allocation . . . 7
  - 3.3.2 N2: Regime Classification . . . 7
  - 3.3.3 N3: Decode Speed in Regime B . . . 7
  - 3.3.4 N4: KV Cache Swap-In Latency . . . 8
  - 3.3.5 N5: TTFT in Regime B . . . 8
  - 3.3.6 N6: TTFT in Regime C . . . 8
  - 3.3.7 N7: TTFT in Regime D . . . 8
  - 3.3.8 N8: Concurrency Limits with KV Offloading . . . 8
  - 3.3.9 N9: Effective Throughput with KV Swapping . . . 9
  - 3.3.10 N10: Quantization vs. Offload Decision Threshold . . . 9

4 Connectivity & Bandwidth Architecture **9**
- 4.1 The Data Path Hierarchy . . . 10
- 4.2 PCIe Bus Architecture . . . 10
  - 4.2.1 PCIe Bandwidth Calculation . . . 10
  - 4.2.2 Practical PCIe Efficiency . . . 10
  - 4.2.3 PCIe Lane Width Impact . . . 11
- 4.3 System RAM Bandwidth . . . 11
  - 4.3.1 Theoretical RAM Bandwidth . . . 11
  - 4.3.2 Practical RAM Efficiency . . . 11
  - 4.3.3 NUMA Effects on Multi-Socket Systems . . . 12
- 4.4 The Transfer Chain: RAM → CPU → PCIe → GPU . . . 12
- 4.5 GPU Internal Architecture & HBM . . . 12
  - 4.5.1 High Bandwidth Memory (HBM) . . . 12
  - 4.5.2 GPU Clock Speed & Compute Throughput . . . 13
  - 4.5.3 The Roofline Model . . . 13
- 4.6 Multi-GPU Interconnectivity . . . 13
  - 4.6.1 NVLink . . . 13
  - 4.6.2 NVSwitch . . . 14
  - 4.6.3 AMD Infinity Fabric . . . 14
  - 4.6.4 PCIe Peer-to-Peer (P2P) . . . 14
- 4.7 Tensor Parallelism vs. Pipeline Parallelism . . . 14
  - 4.7.1 Tensor Parallelism (TP) . . . 14
  - 4.7.2 Pipeline Parallelism (PP) . . . 15
  - 4.7.3 Combined TP + PP . . . 15

4.8 Combined Performance Model . . . 15

5 Performance Estimation **16**
5.1 Decode Speed (Tokens per Second) . . . 16
5.2 Time to First Token (TTFT) . . . 16
5.3 Extended Performance Metrics . . . 16

6 Power & Cost Estimation **16**
6.1 Power Model . . . 16
6.2 Energy and Cost Calculations . . . 17
6.3 Carbon Emissions . . . 17

7 Fine-Tuning Memory Model **17**
7.1 LoRA / QLoRA . . . 17
7.2 Full Fine-Tuning . . . 18

8 Quantization Reference **18**
8.1 GGUF Quantization Levels . . . 18
8.2 Weighted Quantization Methods . . . 18

9 HuggingFace Integration **18**
9.1 Model Metadata Retrieval . . . 18
9.2 Tensor Type Detection . . . 19
9.3 Quantized Variant Discovery . . . 19

10 Hardware Reference **19**
10.1 Supported GPU Hardware . . . 19
10.2 PCIe Bandwidth Reference . . . 19
10.3 Interconnect Latency Reference . . . 19

11 Limitations & Assumptions **19**

12 Glossary **21**

1 Introduction

The LLM VRAM Calculator is a self-contained, browser-based tool designed to estimate the memory requirements, inference performance, and operational costs of deploying Large Language Models (LLMs) on GPU hardware. It integrates directly with the HuggingFace model hub to retrieve real model metadata, including parameter counts, tensor types, quantization configurations, and available quantized variants.

This document provides the complete mathematical foundation, algorithmic descriptions, and pedagogical references for every calculation performed by the tool. Each formula is derived from first principles and annotated with its assumptions, limitations, and practical implications.

1.1 Design Philosophy

The calculator follows three core principles: **rigor** (every estimate is grounded in a well-defined mathematical model), **transparency** (all formulas are displayed to the user with live substitution of their chosen parameters), and **pedagogy** (every input and output is accompanied by an on-demand explanation accessible via the "?" buttons throughout the interface).

1.2 Scope

The calculator addresses two primary deployment scenarios:

- **Inference**: Estimating VRAM requirements for model serving, including weight storage, KV cache, framework overhead, and performance metrics such as tokens-per-second and time-to-first-token.
- **Fine-tuning**: Estimating additional memory for gradient storage, optimizer states (AdamW), and activation memory for both LoRA and full fine-tuning regimes.

2 VRAM Estimation Model

Total GPU memory consumption is modeled as the sum of three primary components:

$$V_{\text{total}} = V_{\text{weights}} + V_{\text{kv}} + V_{\text{overhead}} \tag{1}$$

where V_{weights} is the memory for model parameters, V_{kv} is the KV cache memory, and V_{overhead} accounts for framework and runtime overhead.

2.1 Model Weight Memory

The dominant memory component is the storage of model weights. Its size is determined by the total parameter count and the precision (bytes per parameter):

$$V_{\text{weights}} = P_{\text{total}} \times b_{\text{param}} \tag{2}$$

Definition 2.1 (Total Parameters)**.** P_{total} is the total number of learnable parameters in the model, including *all* experts for Mixture-of-Experts (MoE) architectures. For dense models, $P_{\text{total}} = P_{\text{active}}$. For MoE models, $P_{\text{total}} \gg P_{\text{active}}$ since only a subset of experts is activated per token.

Definition 2.2 (Bytes per Parameter)**.** b_{param} denotes the number of bytes used to store each weight parameter. This depends on the chosen precision or quantization level. Table 1 lists common values.

Table 1: Precision levels and bytes per parameter.

Precision	Bits	B/param	Notes
FP32	32	4.00	Full precision, rarely used for inference
BF16 / FP16	16	2.00	Standard training & inference precision
FP8 / INT8	8	1.00	8-bit quantization
Q8_0	8	1.06	GGUF 8-bit with overhead
Q6_K	6	0.83	6-bit K-quant
Q5_K_M	5	0.71	5-bit medium K-quant
Q4_K_M	4	0.60	4-bit medium K-quant
Q4 / NF4	4	0.50	4-bit quantization, QLoRA default
Q3_K_M	3	0.43	3-bit medium K-quant
Q2_K	2	0.32	2-bit K-quant (aggressive)

2.2 KV Cache Memory

During autoregressive generation, the model caches past Key and Value states from attention layers to avoid recomputation. The KV cache size is:

$$V_{\text{kv}} = 2 \times L \times n_{kv} \times d_h \times C \times B \times U \times b_{kv} \tag{3}$$

where the factor of 2 accounts for separate Key and Value tensors.

Definition 2.3 (KV Cache Parameters)**.**

- L: Number of transformer layers (blocks)
- n_{kv}: Number of key-value heads (depends on attention architecture)
- d_h: Head dimension, typically $d_h = h/n_{\text{heads}}$ or explicitly defined in config
- C: Context length (maximum sequence length in tokens)
- B: Batch size (sequences processed simultaneously)
- U: Number of concurrent users (each requires separate KV cache)
- b_{kv}: Bytes per cached element (2 for FP16, 1 for FP8/INT8, 0.5 for Q4)

2.2.1 Attention Architecture Variants

The number of KV heads n_{kv} varies significantly across attention architectures, directly impacting KV cache memory:

Table 2: Attention architecture variants and their impact on KV cache.

Architecture	KV Heads	Description
Multi-Head (MHA)	$n_{kv} = n_q$	Each query head has its own KV pair. Largest KV cache.
Grouped-Query (GQA)	$1 < n_{kv} < n_q$	Query heads share KV heads in groups. Good balance.
Multi-Query (MQA)	$n_{kv} = 1$	All query heads share a single KV head. Smallest cache.

For example, Llama 3.1 70B uses GQA with $n_q = 64$ query heads but only $n_{kv} = 8$ KV heads, reducing KV cache memory by 8× compared to MHA.

2.2.2 Fallback Estimation

When architecture details (L, n_{kv}, d_h) are unavailable, the calculator falls back to a rule-of-thumb estimate:

$$V_{\text{kv}} \approx P_{\text{total}} \times 0.12 \times \frac{C}{4096} \times B \times U \tag{4}$$

This assumes KV cache is approximately 12% of weight memory at 4096-token context for a typical GQA model.

2.3 Framework Overhead

Framework overhead includes CUDA kernels, temporary buffers, activation memory, and communication buffers. It is estimated as a fraction of weight memory:

$$V_{\text{overhead}} = V_{\text{weights}} \times f_{\text{overhead}} \tag{5}$$

where f_{overhead} varies by operation mode:

Table 3: Overhead factors by operation mode.

Mode	f_{overhead}
Inference	0.12 (12%)
LoRA / QLoRA fine-tuning	0.40 (40%)
Full fine-tuning	2.00 (200%)

For full fine-tuning, the overhead factor of 2.0 accounts for gradients (1× weights in training precision) and AdamW optimizer states (8 bytes per parameter in FP32), plus activation memory.

3 RAM Overflow / Offloading

Performance Warning

RAM offloading allows models that exceed VRAM to run by spilling layers to system RAM, but at a severe performance cost: 10–50× slower inference for the offloaded portion.

When the total required memory exceeds available VRAM, the calculator supports an optional RAM offloading mode:

$$V_{\text{overflow}} = \max(0,\ V_{\text{total}} - V_{\text{VRAM}}) \tag{6}$$

$$V_{\text{RAM,usable}} = \min(V_{\text{overflow}},\ V_{\text{RAM,available}}) \tag{7}$$

The effective available memory becomes:

$$V_{\text{effective}} = V_{\text{VRAM}} + V_{\text{RAM,usable}} \tag{8}$$

3.1 The Bus Wall ("Le Mur du Bus")

The fundamental performance limitation of RAM offloading is captured by the **Bus Wall** concept — the ratio of GPU HBM bandwidth to the effective transfer bandwidth between CPU and GPU:

$$\text{Bus Wall Ratio} = \frac{BW_{\text{HBM}}}{BW_{\text{transfer}}} \tag{9}$$

where BW_{transfer} is the effective bandwidth of the data path from system RAM to GPU, determined by the bottleneck in the transfer chain:

$$BW_{\text{transfer}} = \min(BW_{\text{PCIe,effective}},\ BW_{\text{RAM,effective}}) \tag{10}$$

The Bus Wall ratio tells you how many times slower RAM-offloaded layers are compared to VRAM-resident layers. Typical values range from 30× (RTX 4090 with Gen4 x16) to over 100× (H100 SXM with Gen5 x16).

3.2 Improved RAM Offload Performance Model

The performance degradation from RAM offloading is modeled by decomposing the decode time into two independent data paths:

$$T_{\text{decode}} = \frac{W_{\text{VRAM}}}{BW_{\text{HBM}}} + \frac{W_{\text{RAM}}}{BW_{\text{transfer}}} \tag{11}$$

where W_{VRAM} is the fraction of weights in VRAM and W_{RAM} is the fraction offloaded to RAM. This two-path model is more accurate than a simple degradation factor because it correctly accounts for the fact that VRAM-resident layers still run at full HBM speed, while only the offloaded portion is slowed by the Bus Wall.

The fraction of weights in RAM is:

$$f_{\text{RAM}} = \frac{V_{\text{RAM,usable}}}{V_{\text{total}}} \tag{12}$$

Substituting, the effective decode speed becomes:

$$\text{TPS}_{\text{offload}} = \frac{1}{\frac{(1-f_{\text{RAM}})\times W}{BW_{\text{HBM}}} + \frac{f_{\text{RAM}}\times W}{BW_{\text{transfer}}}} \tag{13}$$

where $W = P_{\text{active}} \times b_{\text{param}}$ is the total weight memory in GB.

3.3 VRAM Priority Allocation & Offloading Regimes

The previous offloading model treated all RAM offloading uniformly, applying the Bus Wall penalty to the entire decode process. However, the two primary components that can be offloaded — model weights and KV cache — have fundamentally different performance implications when offloaded:

- **Weight offloading**: Causes a Bus Wall penalty on *every decode token*, because each token generation requires reading all weights from memory. This is catastrophic for throughput.
- **KV Cache offloading**: Only impacts TTFT (one-time swap-in latency per context switch), because the KV cache is read once at the start of generation and then remains resident during decoding. Decode speed is unaffected.

This distinction leads to a priority-based VRAM allocation model and four distinct performance regimes.

3.3.1 N1: VRAM Priority Allocation

When VRAM is scarce, it must be allocated with careful prioritization. Model weights receive VRAM priority over KV cache because weight offloading causes a per-token Bus Wall penalty, while KV offloading only causes a one-time swap cost:

$$V_{\text{kv,VRAM}}^{\max} = \max(0,\ V_{\text{VRAM}} - V_{\text{weights}} - V_{\text{overhead}}) \tag{14}$$

The actual KV cache stored in VRAM is then:

$$V_{\text{kv,VRAM}} = \min(V_{\text{kv}},\ V_{\text{kv,VRAM}}^{\max}) \tag{15}$$

And the KV cache that must be stored in RAM:

$$V_{\text{kv,RAM}} = \max(0,\ V_{\text{kv}} - V_{\text{kv,VRAM}}^{\max}) \tag{16}$$

If weights alone exceed VRAM, then all VRAM is used for (partial) weights and all KV must go to RAM:

$$V_{\text{weights,RAM}} = \max(0,\ V_{\text{weights}} - V_{\text{VRAM}} + V_{\text{overhead}}) \tag{17}$$

3.3.2 N2: Regime Classification

Based on the allocation, the deployment falls into one of four performance regimes:

Table 4: Offloading regime classification.

Regime	Condition	Description
A	$V_{\text{total}} \leq V_{\text{VRAM}}$	All in VRAM. Full HBM performance.
B	$V_{\text{weights}} \leq V_{\text{VRAM}}$, $V_{\text{kv}} > V_{\text{kv,VRAM}}^{\max}$	Weights in VRAM, KV in RAM. Full decode speed, TTFT + swap.
C	$V_{\text{weights}} > V_{\text{VRAM}}$, $V_{\text{kv}} \leq V_{\text{kv,VRAM}}^{\max}$	Weights in RAM, KV in VRAM. Bus Wall on every token. (Rare)
D	$V_{\text{weights}} > V_{\text{VRAM}}$, $V_{\text{kv}} > V_{\text{kv,VRAM}}^{\max}$	Both in RAM. Bus Wall + KV swap. Worst case.

Key Insight: Why Regime B is dramatically better than C/D

In Regime B, decode speed equals Regime A (full HBM speed) because weights are still read from VRAM. The KV cache swap penalty only affects TTFT, not per-token throughput. In contrast, Regimes C and D impose the Bus Wall on every single decode token, making decode 30–100× slower. This is why the VRAM priority allocation (weights first) is so important.

3.3.3 N3: Decode Speed in Regime B

In Regime B, all weights reside in VRAM, so decode proceeds at full HBM speed:

$$\text{TPS}_B = \text{TPS}_A = \frac{BW_{\text{HBM}}}{P_{\text{active}} \times b_{\text{param}}} \tag{18}$$

This is the critical result: **KV cache offloading does not slow down decode**. Only the initial context loading (TTFT) is impacted.

3.3.4 N4: KV Cache Swap-In Latency

When a user's KV cache is stored in RAM, it must be swapped into VRAM before generation can begin. This is a one-time cost per context switch:

$$T_{\text{kv,swap}} = \frac{V_{\text{kv,RAM}}}{BW_{\text{transfer}}} \tag{19}$$

where $BW_{\text{transfer}} = \min(BW_{\text{PCIe,effective}}, BW_{\text{RAM,effective}})$ is the same transfer bandwidth used in the Bus Wall calculation.

3.3.5 N5: TTFT in Regime B

The time to first token in Regime B adds the KV swap penalty to the standard prefill time:

$$\text{TTFT}_B = \text{TTFT}_A + T_{\text{kv,swap}} \tag{20}$$

This means the first token for a user with offloaded KV cache is delayed by the swap time, but all subsequent tokens generate at full decode speed.

3.3.6 N6: TTFT in Regime C

In Regime C, weight offloading means that the prompt must wait for weights to be loaded from RAM before each layer can compute. The effective TTFT is:

$$\text{TTFT}_C = \max(T_{\text{compute}},\ T_{\text{weight,load}}) \tag{21}$$

where $T_{\text{weight,load}} = V_{\text{weights,RAM}}/BW_{\text{transfer}}$ is the time to load the offloaded weights. In practice, $T_{\text{weight,load}} \gg T_{\text{compute}}$ because the Bus Wall ratio is typically 30–100×.

3.3.7 N7: TTFT in Regime D

Regime D combines the worst of both worlds:

$$\text{TTFT}_D = \max(T_{\text{compute}},\ T_{\text{weight,load}}) + T_{\text{kv,swap}} \tag{22}$$

Both the weight loading penalty and the KV swap penalty apply. This regime should be avoided whenever possible.

3.3.8 N8: Concurrency Limits with KV Offloading

When KV cache is partially stored in RAM, the number of simultaneously served users splits into two groups. Active users have their KV cache entirely in VRAM and experience no swap penalty; swapped users have their KV in RAM and pay the swap cost on context switches:

$$U_{\text{active}} = \left\lfloor \frac{V^{\max}_{\text{kv,VRAM}}}{V_{\text{kv,per user}}} \right\rfloor \tag{23}$$

$$U_{\text{swapped}} = \left\lfloor \frac{V_{\text{RAM,for KV}}}{V_{\text{kv,per user}}} \right\rfloor \tag{24}$$

$$U_{\text{total}} = U_{\text{active}} + U_{\text{swapped}} \tag{25}$$

where $V_{\text{kv,per user}} = 2 \times L \times n_{kv} \times d_h \times C \times b_{kv}/10^9$ is the KV cache size per individual user in GB.

3.3.9 N9: Effective Throughput with KV Swapping

Total throughput with KV swapping accounts for the time spent swapping vs. generating:

$$\text{TPS}_{\text{eff}} = U_{\text{active}} \times \text{TPS} + U_{\text{swapped}} \times \text{TPS} \times \eta_{\text{swap}} \tag{26}$$

where the swap efficiency η_{swap} depends on the ratio of swap time to generation time per context:

$$\eta_{\text{swap}} = \frac{1}{1 + T_{\text{kv,swap}}/T_{\text{gen}}} \tag{27}$$

For long conversations ($T_{\text{gen}} \gg T_{\text{kv,swap}}$), $\eta_{\text{swap}} \approx 1$ and the swap overhead is negligible. For short exchanges, swap overhead is more significant.

3.3.10 N10: Quantization vs. Offload Decision Threshold

When weights do not fit in VRAM, there is a strategic choice: (1) quantize weights more aggressively to fit in VRAM, or (2) keep higher precision but offload to RAM. The Bus Wall makes option 2 almost always inferior:

$$b_{\text{fit}} = \frac{V_{\text{VRAM}} - V_{\text{overhead}}}{P_{\text{total}}} \tag{28}$$

where b_{fit} is the maximum bytes per parameter that allows all weights to fit in VRAM. The decision rule is:

$$\text{Quantize if } f_{\text{RAM}} \times \text{Bus Wall Ratio} > 2 \tag{29}$$

Since typical Bus Wall ratios range from 30–100×, even a small fraction of weights in RAM creates a severe performance penalty. For example, a 70B model at Q4 (35 GB) fits in a single 80 GB A100 with room for KV cache. The same model at FP16 (140 GB) requires offloading 60 GB to RAM, resulting in decode that is ∼50× slower — far worse than any quality loss from Q4 quantization.

Best Practices for Offloading

1. **Quantize weights first**: Reduce b_{param} until $V_{\text{weights}} \leq V_{\text{VRAM}}$. This avoids Regimes C/D entirely.
2. **Quantize KV cache second**: If KV cache still overflows after weight quantization, reduce b_{kv} to FP8 or INT8. This halves or quarters the KV memory.
3. **Use KV offloading for concurrency**: Regime B (KV in RAM, weights in VRAM) is acceptable for multi-user serving because decode speed is preserved.
4. **Hardware matching**: DDR5 + PCIe Gen5 makes KV swap faster (57 GB/s vs 28 GB/s for Gen4), reducing the Regime B penalty.

4 Connectivity & Bandwidth Architecture

This section provides the complete theoretical foundation for the data transfer paths that determine inference performance. Understanding these paths is essential for accurate performance estimation, especially when RAM offloading or multi-GPU configurations are involved.

4.1 The Data Path Hierarchy

Data involved in LLM inference traverses a strict hierarchy of interconnects, each with vastly different bandwidth characteristics:

Table 5: Data path hierarchy for LLM inference.

Path	Interconnect	BW Range	Use Case
GPU HBM	On-die bus	900–4800 GB/s	VRAM-resident weight reading
NVLink/NVSwitch	GPU-GPU link	400–900 GB/s	Tensor Parallelism all-reduce
PCIe	CPU-GPU bus	7–57 GB/s eff.	RAM offload data transfer
System RAM	Memory bus	43–304 GB/s eff.	Offloaded weight storage

The key insight is that each level in this hierarchy is roughly 10–100× slower than the one above it. This creates a "bandwidth cliff" when inference must access slower paths.

4.2 PCIe Bus Architecture

PCI Express (PCIe) is the primary data highway between the CPU and GPU. Its bandwidth is determined by three factors: the generation (signaling rate), the number of lanes (bus width), and practical protocol efficiency.

4.2.1 PCIe Bandwidth Calculation

Theoretical PCIe bandwidth is calculated as:

$$BW_{\text{PCIe,theoretical}} = R_{\text{GT/s}} \times N_{\text{lanes}} \times \frac{128}{130} \div 8 \tag{30}$$

where $R_{\text{GT/s}}$ is the transfer rate per lane, N_{lanes} is the number of lanes (typically 16, 8, or 4), and the 128/130 factor accounts for the encoding overhead introduced in PCIe 3.0+.

Table 6: PCIe bandwidth per generation and lane configuration.

Generation	GT/s/lane	x16 (GB/s)	x8 (GB/s)
PCIe 3.0	8	15.75	7.88
PCIe 4.0	16	31.50	15.75
PCIe 5.0	32	63.00	31.50

4.2.2 Practical PCIe Efficiency

Theoretical bandwidth is never fully achieved. Protocol overhead reduces practical throughput:

$$BW_{\text{PCIe,effective}} = BW_{\text{PCIe,theoretical}} \times \eta_{\text{PCIe}} \tag{31}$$

where $\eta_{\text{PCIe}} \approx 0.90$ for large sequential DMA transfers. The overhead comes from:

- **TLP headers**: Each Transaction Layer Packet has a 12–16 byte header for a 256–4096 byte payload
- **DLLP and ACK/NAK**: Data Link Layer packets add protocol overhead
- **Root complex latency**: CPU-side DMA controller adds a few microseconds of setup latency

- **Memory mapping**: IOMMU address translation for DMA adds overhead

For small random transfers (e.g., individual parameter updates), efficiency can drop to 40–70%. The calculator uses $\eta_{\text{PCIe}} = 0.90$ as a conservative estimate for the large sequential DMA transfers characteristic of weight loading during inference.

4.2.3 PCIe Lane Width Impact

Most desktop and server GPUs connect via x16 (16 lanes). However, some configurations reduce the effective lane width:

- Motherboards that share PCIe lanes between slots may run the GPU at x8 when both slots are populated
- Some budget GPUs are physically x8 or x4
- SXM form factors bypass PCIe entirely for GPU-GPU communication, but still use PCIe for CPU-GPU data transfer

Reducing from x16 to x8 exactly halves the available bandwidth, which significantly impacts RAM offload performance.

4.3 System RAM Bandwidth

4.3.1 Theoretical RAM Bandwidth

System RAM bandwidth is determined by the memory type, transfer rate, and number of channels:

$$BW_{\text{RAM,theoretical}} = MT/s \times N_{\text{channels}} \times 8 \text{ bytes/transfer} \tag{32}$$

Table 7: System RAM bandwidth by type and channel count.

Configuration	MT/s	Channels	BW (GB/s)
DDR4-3200 2-ch	3200	2	51.2
DDR4-3200 4-ch	3200	4	102.4
DDR4-3200 8-ch	3200	8	204.8
DDR5-5600 2-ch	5600	2	89.6
DDR5-5600 4-ch	5600	4	179.2
DDR5-5600 8-ch	5600	8	358.4

4.3.2 Practical RAM Efficiency

As with PCIe, theoretical RAM bandwidth is not fully achievable:

$$BW_{\text{RAM,effective}} = BW_{\text{RAM,theoretical}} \times \eta_{\text{RAM}} \times f_{\text{NUMA}} \tag{33}$$

where $\eta_{\text{RAM}} \approx 0.85$ accounts for DRAM refresh cycles, row misses, and memory controller scheduling, and f_{NUMA} is the NUMA efficiency factor.

4.3.3 NUMA Effects on Multi-Socket Systems

On multi-socket servers (AMD EPYC, Intel Xeon), each CPU socket has its own memory controller and attached RAM. Accessing RAM on the local socket is fast, but accessing RAM attached to a remote socket traverses an inter-socket link (AMD Infinity Fabric or Intel UPI) that adds latency and reduces bandwidth by 30–50%:

$$f_{\text{NUMA}} = \begin{cases} 1.0 & \text{NUMA-aware (weights on local socket)} \\ 0.65 & \text{No NUMA awareness (potential cross-socket access)} \end{cases} \tag{34}$$

The calculator uses $f_{\text{NUMA}} = 0.65$ when NUMA awareness is disabled, reflecting the worst case where weight data may be allocated on a remote socket. For production deployments, always enable NUMA-aware allocation and pin the GPU to the same NUMA node as the RAM holding the offloaded weights.

4.4 The Transfer Chain: RAM → CPU → PCIe → GPU

When RAM-offloaded layers are accessed during inference, data must traverse the entire transfer chain:

1. **RAM read**: Data is read from DRAM through the CPU memory controller
2. **CPU processing**: The CPU's IOMMU maps the DMA buffer address
3. **PCIe DMA transfer**: Data is transferred via DMA from the pinned CPU buffer to GPU VRAM
4. **GPU reception**: The GPU's PCIe controller receives the data into VRAM

The bottleneck in this chain is the slower of PCIe effective bandwidth and RAM effective bandwidth:

$$BW_{\text{transfer}} = \min(BW_{\text{PCIe,effective}},\ BW_{\text{RAM,effective}}) \tag{35}$$

In most configurations, PCIe is the bottleneck. Even DDR4 2-channel at 51.2 GB/s theoretical (≈43 GB/s effective) exceeds PCIe Gen4 x8 at ≈14 GB/s effective. However, with fast PCIe Gen5 x16 (≈57 GB/s effective), slower RAM configurations (DDR4 2-channel) can become the bottleneck instead.

4.5 GPU Internal Architecture & HBM

4.5.1 High Bandwidth Memory (HBM)

GPU HBM is the fastest memory in the inference data path, connected to the GPU die via an ultra-wide bus (3072–6144 bits) on an organic or silicon interposer. This provides bandwidth of 900–4800 GB/s, orders of magnitude faster than any external interconnect.

Table 8: GPU HBM specifications by generation.

Memory Type	Example GPU	Bandwidth	Bus Width
GDDR6X	RTX 4090	1008 GB/s	384-bit
HBM2e	A100	2000 GB/s	5120-bit
HBM3	H100 SXM	3350 GB/s	5120-bit
HBM3e	H200 SXM	4800 GB/s	6144-bit

LLM decode is almost always HBM-bandwidth-bound: each token generation requires reading ALL active weights from HBM, but only performs $2/b$ FLOPs per byte of weight data (where b is bytes per parameter). This arithmetic intensity is far below the GPU's ridge point, confirming the bandwidth-bound nature of decode.

4.5.2 GPU Clock Speed & Compute Throughput

GPU clock speed (typically 1.5–2.5 GHz) affects compute throughput (TFLOPS) but has limited impact on bandwidth-bound inference. The relationship between clock speed and compute throughput is:

$$\text{TFLOPS} = N_{\text{cores}} \times f_{\text{clock}} \times 2 \text{ FLOP/clock/core (tensor cores)} \tag{36}$$

For bandwidth-bound decode, increasing clock speed provides no benefit — the bottleneck is HBM read speed, not compute. Clock speed only matters for compute-bound prefill, where higher TFLOPS directly translates to faster prompt processing.

4.5.3 The Roofline Model

The roofline model provides a unified framework for understanding whether a workload is compute-bound or bandwidth-bound:

Definition 4.1 (Arithmetic Intensity)**.** Arithmetic intensity is the ratio of FLOPs performed to bytes of data accessed:

$$AI = \frac{\text{FLOPs}}{\text{Bytes accessed}} \tag{37}$$

For LLM decode with active parameters P_{active} stored at b bytes per parameter:

$$AI_{\text{decode}} = \frac{2 \times P_{\text{active}}}{P_{\text{active}} \times b} = \frac{2}{b} \tag{38}$$

At Q4 ($b = 0.5$), the arithmetic intensity is only 4 FLOP/byte. At FP16 ($b = 2$), it drops to 1 FLOP/byte. These values are far below the GPU's ridge point (the arithmetic intensity at which compute and bandwidth are equally limiting):

$$AI_{\text{ridge}} = \frac{\text{TFLOPS}}{BW_{\text{HBM}}} \tag{39}$$

For H100: $AI_{\text{ridge}} = 990/3350 \approx 0.30$ TFLOPS/(GB/s) $= 295$ FLOP/byte. This confirms that decode is deeply bandwidth-bound regardless of quantization level.

For prefill, the arithmetic intensity is much higher because the same weights are reused across all prompt tokens in a single batched matrix multiplication. Prefill is typically compute-bound.

4.6 Multi-GPU Interconnectivity

4.6.1 NVLink

NVLink is NVIDIA's proprietary high-speed GPU-to-GPU interconnect. It provides dramatically higher bandwidth than PCIe, enabling efficient Tensor Parallelism (TP) where model weights are split across multiple GPUs.

NVLink uses a point-to-point topology: each GPU has direct links to specific other GPUs. In a 4-GPU system, this creates a mesh where each GPU connects to every other GPU. In an 8-GPU system, each GPU typically connects to 4 neighbors, and multi-hop routing is required for non-adjacent communication.

Table 9: NVLink specifications by generation.

Generation	Links	BW per GPU	Example GPU
NVLink 2	6	300 GB/s	V100
NVLink 3	12	600 GB/s	A100
NVLink 4	18	900 GB/s	H100/H200

4.6.2 NVSwitch

NVSwitch is NVIDIA's switching fabric that provides full all-to-all connectivity between all GPUs in a node. Unlike point-to-point NVLink, NVSwitch allows any GPU to communicate with any other GPU at full NVLink speed simultaneously.

$$\text{All-reduce}_{\text{NVSwitch}} = 2 \times \frac{h \times 2}{BW_{\text{NVLink}}} \tag{40}$$

vs. ring all-reduce on point-to-point NVLink:

$$\text{All-reduce}_{\text{ring}} = 2 \times \frac{N-1}{N} \times \frac{h \times 2}{BW_{\text{NVLink}}} \tag{41}$$

where h is the hidden size and N is the number of GPUs. NVSwitch eliminates the $(N-1)/N$ penalty and reduces the number of hops, providing significantly better TP efficiency for 4+ GPUs.

4.6.3 AMD Infinity Fabric

AMD's Infinity Fabric serves a similar role to NVLink on MI-series GPUs. The MI300X is a multi-chiplet accelerator integrating 8 compute dies (XCDs) and 4 I/O dies — 12 chiplets in total — connected via AMD Infinity Fabric within the package. It provides 192 GB of HBM3 memory at 5,300 GB/s aggregate bandwidth. For multi-GPU scaling, each discrete MI300X offers a 16-lane PCIe® Gen 5 host interface and seven external AMD Infinity Fabric links (each at 128 GB/s bidirectional), allowing full all-to-all connectivity between eight GPUs in a ring topology.

4.6.4 PCIe Peer-to-Peer (P2P)

Without NVLink or Infinity Fabric, GPUs can communicate via PCIe peer-to-peer transfers. This uses the PCIe bus for direct GPU-to-GPU communication without CPU involvement, but the bandwidth is limited to PCIe speeds (typically 15–57 GB/s effective), making TP inefficient for all but the smallest models.

4.7 Tensor Parallelism vs. Pipeline Parallelism

4.7.1 Tensor Parallelism (TP)

TP splits model weights across N GPUs. Each GPU holds $1/N$ of the weights and performs the corresponding portion of each matrix multiplication. After each transformer layer, an all-reduce synchronizes the partial results across all GPUs.

The decode time per token with TP is:

$$T_{\text{TP}} = \frac{P_{\text{active}} \times b}{N \times BW_{\text{HBM}}} + L \times \left(\frac{2 \times \frac{N-1}{N} \times h \times 2}{BW_{\text{interconnect}}} + \lambda_{\text{AR}} \right) \tag{42}$$

where λ_{AR} is the all-reduce latency per layer (5–100 μs depending on interconnect).

TP efficiency is:

$$\eta_{\text{TP}} = \frac{T_{\text{compute}}}{T_{\text{compute}} + T_{\text{communication}}} \tag{43}$$

Table 10: Typical TP efficiency by interconnect type.

Interconnect	**2-GPU**	**4-GPU**	**8-GPU**
NVSwitch (H100)	96%	93%	88%
NVLink P2P (H100)	94%	87%	75%
PCIe Gen4 x16	65%	40%	20%

TP is the preferred strategy when NVLink or NVSwitch is available, as it provides near-linear scaling with minimal latency overhead.

4.7.2 Pipeline Parallelism (PP)

PP splits model layers across GPUs sequentially. Each GPU processes a contiguous group of layers and passes the activation tensor to the next GPU. PP requires much less communication than TP — only the activation tensor (hidden size × batch × precision) is sent between stages, not an all-reduce.

However, PP introduces pipeline bubbles — idle time where GPUs wait for preceding stages to complete:

$$\text{Bubble fraction} = \frac{N - 1}{N + M - 1} \tag{44}$$

where M is the number of micro-batches. For single-user inference with batch size 1, only 1 micro-batch is possible, giving bubble fraction $(N - 1)/N$.

PP is preferred when NVLink is unavailable (e.g., consumer GPUs connected via PCIe) or for cross-node deployment where network latency makes TP impractical.

4.7.3 Combined TP + PP

For very large models on many GPUs, both strategies can be combined: TP within a node (using NVLink) and PP across nodes (using network). For example, a 405B model on 8 H100s might use TP=4 within two nodes and PP=2 across nodes.

4.8 Combined Performance Model

The complete decode time model accounts for all connectivity factors:

$$T_{\text{decode}} = \underbrace{\frac{(1 - f_{\text{RAM}}) \times W}{\eta_{\text{TP}} \times N \times BW_{\text{HBM}}}}_{\text{VRAM-resident weights (TP-accelerated)}} + \underbrace{\frac{f_{\text{RAM}} \times W}{\min(\eta_{\text{PCIe}} \times BW_{\text{PCIe}},\ \eta_{\text{RAM}} \times f_{\text{NUMA}} \times BW_{\text{RAM}})}}_{\text{RAM-offloaded weights (Bus Wall limited)}} \tag{45}$$

This formula captures the essential physics of LLM inference:

- VRAM-resident layers benefit from both HBM bandwidth and TP parallelism
- RAM-offloaded layers are limited by the slowest link in the transfer chain
- The Bus Wall ratio quantifies the performance gap between these two regimes

- NUMA effects can further degrade RAM-offloaded performance on multi-socket systems
- PCIe lane width and generation directly affect the transfer bottleneck

5 Performance Estimation

5.1 Decode Speed (Tokens per Second)

During autoregressive decoding, each token generation requires loading all model weights from GPU memory. This makes inference **bandwidth-bound**:

$$\text{TPS} = \frac{BW}{P_{\text{active}} \times b_{\text{param}} \times 1000} \tag{46}$$

where BW is the GPU memory bandwidth in GB/s. For MoE models, P_{active} (the number of parameters active per forward pass) is used instead of P_{total}, since only the routed experts are loaded during decode.

Definition 5.1 (Active Parameters)**.** For dense models, $P_{\text{active}} = P_{\text{total}}$. For MoE models, P_{active} represents only the parameters used per forward pass, including the embedding layer, shared experts, and the k routed experts per token. For example, Mixtral 8x7B has $P_{\text{total}} = 47B$ but $P_{\text{active}} \approx 13B$.

5.2 Time to First Token (TTFT)

The prefill phase processes the entire prompt in parallel. The time to first token is estimated as:

$$\text{TTFT} = \frac{P_{\text{active}} \times C \times 2 \times b_{\text{param}}}{BW} \times 1000 \text{ ms} \tag{47}$$

where C is the prompt length in tokens. The factor of 2 accounts for the read and write of activations during the forward pass.

5.3 Extended Performance Metrics

The calculator provides additional derived metrics for practical deployment planning:

$$\text{Latency per token} = \frac{1000}{\text{TPS}} \text{ ms} \tag{48}$$

$$\text{Time}_{100} = \frac{\text{TTFT}}{1000} + \frac{100}{\text{TPS}} \text{ seconds} \tag{49}$$

$$\text{Time}_{1000} = \frac{\text{TTFT}}{1000} + \frac{1000}{\text{TPS}} \text{ seconds} \tag{50}$$

$$\text{Throughput} = \text{TPS} \times U \text{ tok/s total} \tag{51}$$

6 Power & Cost Estimation

6.1 Power Model

GPU power consumption is estimated from the Thermal Design Power (TDP) and utilization:

$$P_{\text{draw}} = \text{TDP} \times \frac{U_{\text{GPU}}}{100} \tag{52}$$

where U_{GPU} is the GPU utilization percentage. LLM inference is typically memory-bandwidth-bound rather than compute-bound, resulting in 60–90% utilization during decode.

6.2 Energy and Cost Calculations

$$E_{\text{hour}} = \frac{P_{\text{draw}}}{1000} \text{ kWh} \tag{53}$$

$$C_{\text{hour}} = E_{\text{hour}} \times R_{\text{elec}} \tag{54}$$

$$C_{\text{day}} = C_{\text{hour}} \times H_{\text{day}} \tag{55}$$

$$C_{\text{month}} = C_{\text{day}} \times 30 \tag{56}$$

$$C_{\text{1M tok}} = \frac{C_{\text{hour}}}{\text{TPS} \times 3600} \times 10^6 \tag{57}$$

where R_{elec} is the electricity rate (\$/kWh), H_{day} is operating hours per day, and TPS is the decode speed.

6.3 Carbon Emissions

$$\text{CO}_{2,\text{hour}} = E_{\text{hour}} \times I_{\text{carbon}} \tag{58}$$

$$\text{CO}_{2,\text{annual}} = \text{CO}_{2,\text{hour}} \times H_{\text{day}} \times 365/1000 \text{ tonnes} \tag{59}$$

where I_{carbon} is the grid carbon intensity in kg CO_2/kWh. Default values and regional references:

Table 11: Grid carbon intensity by region.

Region	**kg CO_2/kWh**
World average	0.417
European Union	0.255
United States	0.387
France (nuclear)	0.056
Sweden (hydro)	0.045
Poland (coal)	0.769
China	0.555

7 Fine-Tuning Memory Model

For fine-tuning, the memory model extends to include gradients and optimizer states:

$$V_{\text{FT}} = V_{\text{weights}} + V_{\text{gradients}} + V_{\text{optimizer}} + V_{\text{activations}} \tag{60}$$

7.1 LoRA / QLoRA

With LoRA, only low-rank adaptation matrices are trained. The additional memory is:

$$V_{\text{LoRA}} \approx 0.40 \times V_{\text{weights}} \tag{61}$$

This accounts for LoRA weights (typically $< 1\%$ of base weights), their gradients (FP32), and 8-bit AdamW states.

7.2 Full Fine-Tuning

Full fine-tuning requires gradients for all parameters and optimizer states:

$$V_{\text{gradients}} = P_{\text{total}} \times b_{\text{train}} \tag{62}$$

$$V_{\text{optimizer}} = P_{\text{total}} \times 8 \text{ (AdamW FP32: momentum + variance)} \tag{63}$$

Combined with the overhead factor of 2.0, this yields approximately 3× the weight memory for FP16 training with AdamW.

8 Quantization Reference

8.1 GGUF Quantization Levels

GGUF (GPT-Generated Unified Format) is the standard format for llama.cpp and compatible engines. The following table lists supported quantization levels:

Table 12: GGUF quantization levels with bits-per-byte and descriptions.

Level	Label	B/param	Description
Q2_K	2-bit K-quant	0.32	Aggressive 2-bit quantization, K-quants method
Q3_K_S	3-bit small	0.34	3-bit quantization, small variant
Q3_K_M	3-bit medium	0.43	3-bit quantization, medium variant
Q3_K_L	3-bit large	0.45	3-bit quantization, large variant
Q4_0	4-bit base	0.56	Basic 4-bit quantization
Q4_K_S	4-bit small K	0.58	4-bit K-quant, small variant
Q4_K_M	4-bit medium K	0.60	4-bit K-quant, medium variant
Q5_0	5-bit base	0.68	Basic 5-bit quantization
Q5_K_S	5-bit small K	0.69	5-bit K-quant, small variant
Q5_K_M	5-bit medium K	0.71	5-bit K-quant, medium variant
Q6_K	6-bit K-quant	0.83	6-bit K-quant
Q8_0	8-bit quant	1.06	Near-FP16 quality, 8-bit quantization

8.2 Weighted Quantization Methods

Beyond GGUF, the calculator recognizes several weighted quantization formats commonly found on HuggingFace:

9 HuggingFace Integration

9.1 Model Metadata Retrieval

The calculator fetches model metadata from the HuggingFace API using the endpoint:

```
GET https://huggingface.co/api/models/{model_id}
```

This returns the model card data including parameter counts (from `safetensors.total`), tensor types (from `safetensors.parameters`), siblings (file list), and default branch information.

Table 13: Weighted quantization methods.

Method	Typical Bits	Characteristics
GPTQ	2–8 bits	Post-training quantization with calibration dataset; group-wise quantization with optional desc_act; commonly 4-bit with group_size=128
AWQ	4–8 bits	Activation-aware weight quantization; preserves salient weights; group-wise with zero-point
EXL2	2–8 bits	ExLlamaV2 format; mixed-precision per-layer quantization; optimized for ExLlamaV2 inference engine
BNB/NF4	4 bits	BitsAndBytes NF4 quantization; default for QLoRA; double quantization support

9.2 Tensor Type Detection

Tensor type detection follows a 4-level priority chain:

1. **quantization_config** in `config.json`: Highest priority for explicitly quantized models (GPTQ, AWQ, EXL2, BNB).
2. **safetensors.parameters** from the API: Shows actual dtype names of stored tensors (e.g., `F16`, `Q8_0`).
3. **Safetensors index metadata**: dtype fields in `model.safetensors.index.json`.
4. **torch_dtype** in `config.json`: Fallback for unquantized models (typically `bfloat16` or `float16`).

9.3 Quantized Variant Discovery

The calculator automatically discovers quantized variants by:

1. Scanning current model's siblings for GGUF files (parsing filenames like `model-Q4_K_M.gguf`).
2. Searching HuggingFace for related repos: `[base-name] GGUF`, `[base-name] GPTQ`, `[base-name] AWQ`.
3. Deduplicating and sorting variants by quantization level.

10 Hardware Reference

10.1 Supported GPU Hardware

10.2 PCIe Bandwidth Reference

10.3 Interconnect Latency Reference

11 Limitations & Assumptions

1. **Roofline model**: Performance estimates assume bandwidth-bound decode (confirmed by arithmetic intensity analysis). Actual performance may be compute-bound for very small models or very short sequences with high batch sizes.
2. **PCIe efficiency**: The 0.90 efficiency factor is a conservative estimate for large DMA transfers. Real efficiency varies by motherboard, IOMMU configuration, and CPU architecture. Measured values range from 0.85 to 0.95.

Table 14: Supported GPU hardware specifications.

GPU	**VRAM** (GB)	**HBM BW** (GB/s)	**TFLOPS** (Dense)	**TFLOPS** (Sparse)	**PCIe** Gen	**NVLink** (GB/s)	**NVS**
H200 SXM	141	4800	989	1979	5	900	Yes
H100 SXM	80	3350	989	1979	5	900	Yes
H100 PCIe	80	2000	756	1513	5	0	No
A100 80GB	80	2000	312	624	4	600	Yes
A100 40GB	40	1555	312	624	4	600	Yes
A6000 Ada	48	960	182	364	4	0	No
RTX 4090	24	1008	165	330	4	0	No
RTX 3090	24	936	71	142	4	0	No
L40S	48	864	366	733	4	0	No
MI300X	192	5300	1307	2614	5	400	No
MI250X	128	3276	383	383	4	400	No

*Sparse TFLOPS assume a 2:4 structured sparsity pattern. Older architectures like MI250X do not feature structured sparsity.

Table 15: PCIe bandwidth by generation, lane width, and effective throughput.

Config	**Theor.**	**Eff. (η=0.90)**	**Wall (H200)**	**Wall (4090)**
Gen3 x16	15.75 GB/s	14.2 GB/s	338×	71×
Gen3 x8	7.88 GB/s	7.1 GB/s	676×	142×
Gen4 x16	31.5 GB/s	28.4 GB/s	169×	36×
Gen4 x8	15.75 GB/s	14.2 GB/s	338×	71×
Gen5 x16	63.0 GB/s	56.7 GB/s	85×	18×
Gen5 x8	31.5 GB/s	28.4 GB/s	169×	36×

Table 16: Typical all-reduce latency per layer by interconnect type.

Interconnect	**Latency**	**Topology**	**Notes**
NVSwitch	∼5 μs	All-to-all	Best for 4+ GPUs
NVLink P2P	∼10 μs	Ring/mesh	Direct GPU-GPU link
PCIe P2P	∼40 μs	Ring	Through root complex
Through CPU	∼100 μs	Multi-hop	GPU → CPU → GPU

3. **RAM efficiency**: The 0.85 factor for sequential RAM reads is an average. Actual values depend on DRAM type, frequency, and access pattern. Mixed read/write workloads achieve lower efficiency.

4. **NUMA model**: The 0.65 factor for non-NUMA-aware allocation is an approximation. Actual cross-socket bandwidth depends on the inter-socket link (AMD Infinity Fabric, Intel UPI), memory topology, and system firmware configuration.

5. **KV cache quantization**: Quality impact of aggressive KV cache quantization (Q4) is not modeled; it may degrade output quality for sensitive tasks.

6. **RAM offloading**: The regime-aware model (N1–N10) distinguishes weight offloading from KV cache offloading, but still simplifies the real behavior of offloading engines. In practice, llama.cpp and vLLM may use pipelined or overlapped transfers that partially hide latency. The KV swap model assumes sequential swap-in, while real implementations may prefetch or stream KV cache during prefill.

7. **Multi-GPU TP**: Estimates use the analytical all-reduce model with fixed latency. Real implementations may use custom all-reduce algorithms (e.g., NVLink SHARP, NCCL topology-aware) that differ from the ring model.

8. **Pipeline Parallelism**: The bubble fraction model assumes single micro-batch inference. With continuous batching or multiple concurrent requests, the bubble can be hidden more effectively.

9. **Framework overhead**: The 12% inference overhead is a conservative average. vLLM and llama.cpp may use less; PyTorch native may use more.

10. **MoE active parameters**: The calculator uses the active parameter count reported by the model card or estimated from config. Actual activation patterns may differ, and expert routing is token-dependent.

11. **Power model**: TDP-based power estimation is approximate. Real power varies with clock speed, temperature throttling, and workload characteristics.

12. **GPU clock speed**: Clock speed is not explicitly modeled in the calculator since decode is bandwidth-bound. For compute-bound prefill, the TFLOPS value implicitly captures clock speed effects.

12 Glossary

VRAM
: Video Random Access Memory – the dedicated high-bandwidth memory on a GPU, used to store model weights, activations, and KV cache during inference.

HBM
: High Bandwidth Memory – a type of VRAM connected to the GPU die via an ultra-wide bus (3072–6144 bits) on an interposer, providing 900–4800 GB/s bandwidth.

KV Cache
: Key-Value Cache – stored attention key and value tensors from previous positions, enabling efficient autoregressive generation without recomputing the entire context at each step.

GQA
: Grouped-Query Attention – an attention mechanism where query heads are divided into groups, each sharing a single KV head. Balances MHA quality with MQA efficiency.

MQA
: Multi-Query Attention – all query heads share a single KV head, minimizing KV cache size at the cost of some expressiveness.

MoE
: Mixture of Experts – a model architecture where only a subset (typically 2–8) of expert modules is activated per token, reducing compute while maintaining model capacity.

TPS
: Tokens Per Second – the generation speed during autoregressive decoding, the primary throughput metric for LLM inference.

TTFT
: Time to First Token – the latency from receiving a prompt to generating the first output token, dominated by the prefill (prompt processing) phase.

TDP
: Thermal Design Power – the maximum sustained power dissipation a cooling system must handle, used as a proxy for peak GPU power consumption.

QLoRA
: Quantized Low-Rank Adaptation – a parameter-efficient fine-tuning method that quantizes the base model to 4-bit (NF4) and trains small low-rank adapter matrices.

GGUF
: GPT-Generated Unified Format – a file format for storing quantized LLM weights, designed for efficient loading and inference with llama.cpp and compatible engines.

Bus Wall
: Le Mur du Bus – the ratio of GPU HBM bandwidth to the effective transfer bandwidth (PCIe + RAM) for RAM-offloaded layers. Quantifies how many times slower offloaded layers are compared to VRAM-resident layers.

PCIe
: Peripheral Component Interconnect Express – the primary data bus between CPU and GPU, providing 7–57 GB/s effective bandwidth depending on generation and lane count.

NVLink
: NVIDIA's proprietary high-speed GPU-to-GPU interconnect, providing 300–900 GB/s bandwidth for Tensor Parallelism communication.

NVSwitch
: NVIDIA's switching fabric providing all-to-all connectivity between GPUs in a node, improving TP efficiency for 4+ GPUs compared to point-to-point NVLink.

NUMA
: Non-Uniform Memory Access – a memory architecture where each CPU socket has local memory; accessing remote socket memory is slower, affecting RAM offload performance on multi-socket servers.

Arithmetic Intensity
: FLOPs performed per byte of data accessed, determining whether a workload is compute-bound or bandwidth-bound according to the roofline model.

Ridge Point
: The arithmetic intensity at which a GPU transitions from bandwidth-bound to compute-bound execution, equal to $\text{TFLOPS}/\text{BW}_{\text{HBM}}$.

TP Tensor Parallelism – a multi-GPU strategy that splits model weights across GPUs, requiring all-reduce communication after each layer.

PP Pipeline Parallelism – a multi-GPU strategy that splits model layers across GPUs sequentially, requiring only activation tensor communication between stages.